EHB415 ISAC projesi: görev kartları 2 (ikinci faz)

Birinci fazda üç aracın kurulumunu doğruladık (birleşik rapor). Bu fazda her araçtan *sayı* üretiyoruz. Önce Ortak bilgi'yi, sonra kendi kartını oku.

Proje sorusu ve ortak eksen

Soru: sensing'e ayrılan kaynak payı α arttıkça sensing ne kazanır, haberleşme ne kaybeder? Sionna kazancı, ns-3 ve OCUDU kaybı ölçer. Ortak eksen α (sensing'e giden kaynağın toplam kaynağa oranı). Standartta (TR 38.765 §6.2) kaynak oranı Tip 1/2/3 ile tanımlıdır; Dickerson vd. modeli: $\alpha_{sense} = \frac{N_b}{1-OH} \cdot \frac{f_{sense}}{N_{sym/sn}} \cdot \frac{N_{PRB,sense}}{N_{PRB,toplam}}$ ve $T(\eta) = \min\{\eta, 1 - \alpha_{sense}\}$.

Kartlardaki işaretler ve kurallar

- **[Okundu]**: o bölümü kartı hazırlayan kişi PDF'ten ya da kaynak koddan okudu; kartta yazanlar o metinde ya da kodda görülenlerdir.
- **[Yalnızca özet]**: yalnızca özetini gördük; tam metni sen okuyacaksın, kartta onun hakkında özette yazandan fazlası yok.
- **Kontrol kutusu**: komutu bu bilgisayarda çalıştırıp gördüğümüz sayılar. Sende farklı çıkarsa **farkı yaz**; hata sayılmaz.
- Teslimde **çalıştırdığın komutu** ve **gördüğün çıktıyı** yaz. Anlamadığın yeri "anlamadığım yer" diye yaz.
- Yeni dosyalar için: `git pull`. Yeni betikler: `ogretici/betikler/ns3\_yuk\_sweep.sh`, `ns3\_bant\_sweep.sh`, `ocudu\_prs\_kos.sh` (yeni 5. argüman), `ogretici/sionna\_sehir/tr\_metrik\_olc.py`; veriler `ogretici/veri/`.

Kim	Alan	Bu fazda	Çıktı
Taha	3GPP, metrik, rapor	TR 38.765 §6.3 sonuçları; α modelini OCUDU ve ns-3 ölçümleriyle karşılaştırma	3 tablo + rapor bölümleri
Nihat	Sionna	TR metrikleri (Pf1, Pmd hedefi), X_{SI} deneyi, varsayım karşılaştırması	Çıktı tablosu + karşılaştırma tablosu
Aziz	ns-3 / 5G-LENA	Yük ve bant taraması, Sionna-RT örneğinin durumu, "hangi yol" kararı	2 tablo + 1 sayfa karar notu
Fatih	OCUDU	PRS ofset deneyi, kod okuma, çok kullanıcılı deney	Tablo + 2 sorunun cevabı

Teslim tarihi: _____ (kartı verirken yaz)

Taha: 3GPP sonuçları, metrik ve α doğrulaması

Bu görev neden önemli?

Rapor iki şeyi gösterecek: standardın kaynak oranı ve sonuçları nasıl verdiği, ve bizim α tanımımızın ölçümle tutup tutmadığı. TR 38.765 sonuçlarını **sensing kaynak oranına göre iki sınıfa** ($\leq$ %10 ve $>$ %10) ayırıyor (Tablo 6.3-1 ve 6.3-2); bizim α ekseni tam bu sınıflandırmaya oturuyor.

Okuma

Kaynak	Yer	Ne arayacaksın	Durum
3GPP TR 38.765 v20.0.0 (`makaleler/`)	§6.2 Tablo 6.2-1; §6.3.0; §6.3.2 Tablo 6.3-2 ve Case 2-1…2-6 (sayfa 15–20)	Baseline 2 (37 dBm) bizim senaryomuz. Tablo 6.3-2: 6 durum; sütunlar: *kaç TRP*, *sensing kaynak oranı* ($\leq$ %10 / $>$ %10), *Tx ışın sayısı*. Her durumda P_{md}, P_{f1}, P_{f2}, konum ve hız aralıklarını not et. Tek TRP = bizim monostatik gNB.	[Okundu]
	§5.1	Ölçüm raporu seviyeleri A, B, C, D (ham veriden nesne düzeyine); her seviyenin neyi taşıdığı.	[Okundu]
Dickerson vd. (arXiv 2606.21677)	III.B (denklem 9–12), IV.A, IV.D	IV.D: iki sabit politika: sensing-öncelikli %15 ayırır (throughput sınırı 0,85), haberleşme-öncelikli %3 ayırır (sınır 0,97). Bu iki sayı, $T = 1 - \alpha$ kuralının makaledeki örneği.	[Okundu]
Blandino vd., *Evaluation of gNB Monostatic Sensing for UAV Use Case* (2026, arXiv; Consensus ile bulundu)	Özet	3GPP UMa-AV varsayımlarıyla gNB monostatic sensing; özet: PRS ve CP-OFDM kullanıyor, %5'ten az yanlış alarmla %70'ten fazla tespit, 90. yüzdelik hata dikeyde 4 m, yatayda 6 m; simülatör `5GNRad` yayınlanmış. TR 38.765 sonuçlarıyla aynı senaryo olduğu için raporda karşılaştırma kaynağı.	[Yalnızca özet]

Adımlar

1. **TR Baseline 2 sonuç tablosu** (`docs/metrics.md`'ye yeni bölüm). Satırlar Case 2-1...2-6; sütunlar: TRP sayısı, kaynak oranı, ışın, kaynak sayısı, P_{md} aralığı, P_{f2} aralığı, hedefler tuttu mu (evet / hayır). Değerleri §6.3.2'den kendin yaz.
2. α **modelini OCUDU ölçümüyle karşılaştır.** Veri: `ogretici/veri/ocudu\_prs\_sweep.csv` (PDSCH, logda `prb=[0,96)` olarak görülüyor, yani 96 PRB). Her PRS satırı için model kaybı:

$$\text{kayıp}_{model} = \frac{1}{T_{PRS}} \cdot \frac{\min(bw, 96)}{96}, \qquad \text{kayıp}_{llen} = \frac{59{,}0 - \text{dl_brate}}{59{,}0}$$

   Tablo: etiket, T_{PRS}, bw, model %, ölçülen %. Excel ya da Python.
3. α **modelini ns-3 ölçümüyle karşılaştır** (Aziz'in `ns3\_bant.csv` dosyasından; Aziz üretince ya da `ogretici/veri/ns3\_bant.csv`). $C(\alpha)/C_0$ ile $1 - \alpha$ yan yana; C_0 = bant daraltmasız doygun throughput.
4. **Tip 1/2/3 yorumu.** `docs/metrics.md` bölüm 4'teki OCUDU satırını kontrol et: ölçümde PRS 52 PRB $\times$ 12 sembol; Tip 1 = PRS'in kapladığı kaynak. Aziz'in ve Fatih'in bulgularına göre (kartlarında) Tip 3 satırını güncelle.
5. **Rapor 2 taslağı:** Yöntem (üç araçta α) ve Ön bulgular (3 tablo: adım 1, 2, 3). Ders raporu biçimi: birleşik rapor PDF'i (Taha'da).

Kontrol: bizde ölçülen sonuç (kendi sonucunla karşılaştır)

Adım 2 (OCUDU): p4 ($T = 4$, 104 PRB): model %25,00, ölçülen %25,76. p20: %5,00 ve %5,08. p10: %10,00 ve %10,17. p8: %12,50 ve %13,73. p4w52 ($T = 4$, 52 PRB): %13,54 ve %13,56. p80: %1,25 ve %1,69; p40: %2,50 ve %2,03.
Adım 3 (ns-3, 1 band, 2 UE, doygun BE akışı): $C_0 = 164{,}54$ Mbps. $\alpha = 0{,}20$: 133,41 ($C/C_0 = 0{,}811$); $\alpha = 0{,}40$: 102,54 (0,623); $\alpha = 0{,}50$: 77,06 (0,468). $\alpha = 0{,}03$ (48,5 MHz): 164,54 (oran 1,000); $\alpha = 0{,}15$ (42,5 MHz): 143,77 (0,874). $\alpha = 0{,}80$ (10 MHz): 0 paket alındı.
Adım 1: Case 2-4 (tek TRP, $\leq$ %10, 1x1): P_{md} 16,0% ve 12,6–23,0%; hedefler tutmuyor. Case 2-6 (tek TRP, $>$ %10, 1x1): 1 sonuçta P_{md} 1,30%, 3 sonuçta 12,30–37,80%. Case 2-2 (çok TRP, $\leq$ %10, 1x2/1x6/2x4): P_{md} 1,27–4,67%, tüm hedefler tutuyor.

Ne teslim edeceksin?

(1) Baseline 2 sonuç tablosu; (2) OCUDU model–ölçüm tablosu ve 3 cümlelik yorum; (3) ns-3 model–ölçüm tablosu ve yorum; (4) güncellenmiş Tip 1/2/3 satırı; (5) Rapor 2 Yöntem ve Ön bulgular taslağı.

Nihat: Sionna (TR metrikleri ve varsayım karşılaştırması)

Bu görev neden önemli?

TR 38.765 başarıyı $P_{md} \leq \%5$, $P_{f1} \leq \%5$, $P_{f2} \leq \%5$ ve konum hatası $\leq$10 m (%90) ile ölçüyor (§4.2). Elimizdeki radar bugüne kadar yalnızca P_d ve RMSE verdi. Bu kart radarı standardın metrikleriyle okutuyor ve varsayımlarımızı standart ve literatürle yan yana koyuyor.

Okuma

Kaynak	Yer	Ne arayacaksın	Durum
Blandino vd. (arXiv 2505.24763), `makaleler/`	III.A (denklem 9), III.C, III.D (denklem 15), IV.A (Tablo I), IV.B	III.A: statik clutter, PRS sembolleri boyunca ortalama çıkarılarak bastırılıyor. III.C: menzil için IDFT. III.D: tespit istatistiği = menzil tepesi / menzil kutularının ortalaması (PAR); eşik IV.B'de P_{FA} ve P_D eğrilerinden seçilmiş (3,4 dB). IV.A: 30 GHz, 120 kHz, 66 RB, comb $K = 4$, 4 PRS sembolü, 256 PRS, 32×32 dizi, EIRP 75 dBm/100 MHz, RCS $-12{,}81$ dBsm, 4000 hedef atışı.	[Okundu]
Lin (arXiv 2608.11606)	IV.B	Monostatic sensing'in 5 adımı (sinyal yeniden oluşturma, kalibrasyon ve öz-girişim bastırma, gecikme-Doppler-açı işleme, clutter bastırma ve tespit, parametre kestirimi).	[Okundu]
Khosroshahi vd. (arXiv 2408.00667)	II.A	PRS: comb 2/4/6/12 ve sembol 1/2/4/6/12 seçenekleri; PRS boş RE bırakıyor ve bu menzil belirsizliği yaratıyor.	[Okundu]
Riihonen vd., *Full-Duplex OFDM Radar With LTE and 5G NR Waveforms* (2019, IEEE TMTT; Consensus)	Özet	Verici–alıcı izolasyonu sınırlı olduğunda sorun esas olarak statik hedeflerin tespitinde; hareketli hedefler verici öz-girişimine karşı doğası gereği daha dayanıklı.	[Yalnızca özet]

Adımlar

1. **Ön koşul:** `rt_sehir.py` ve `radar_takip.py` bir kez çalışmış olmalı (`cikti/rt\_sonuclar.npz` ve `cikti/kaynak\_basari.csv` oluşur). Python'a `pandas` kurulu olmalı.
2. **Lin'in 5 adımını kodla eşle.** `radar\_takip.py` içinde her adımın hangi satırda olduğunu bul. Yazılacak tablo: *Lin adımı | satır | var / yok*. Bizim modelde clutter yok (`radar()` yalnızca hedef ve gürültü üretiyor); bu satırı "yok" yaz ve III.A denklem 9'un ne yaptığını bir cümleyle yaz.
3. **TR metriklerini ölç:**

```
$env:DRJIT_LIBLLVM_PATH = [Environment]::GetEnvironmentVariable('DRJIT_LIBLLVM_PATH','User')
cd ogretici\sionna_sehir
C:\Users\<kullanici>\venvs\ehb415\Scripts\python.exe tr_metrik_olc.py      # bizde 79 sn surdu
```

   Ubuntu'da: `~/venvs/ehb415/bin/python tr_metrik_olc.py`. Betik üç şey yazıyor: P_{f1} (hedefsiz 3000 CPI), her menzil için $P_{md} \leq \%5$ sağlayan en küçük α, $X_{SI} = 40$ ve 30 dB için 300 m'de P_d.
4. **Konum hatasının %90 yüzdeliği.** `radar\_takip.py`'deki `kaynak_basari()` içinde `hr` (menzil hatası) ve `hv` (hız hatası) listeleri dolduruluyor. Her $(menzil, M)$ için `np.percentile(np.abs(hr), 90)` ve hız için aynısını yazdıran bir satır ekle. TR hedefleri: konum 10 m, hız 5 m/s (%90).
5. X_{SI} **sonucunu açıkla.** Betik çıktısındaki 3. bölüme bak. TR 38.765 Ek A (Tablo A-1, not 2): artık sızıntı "$-94 + X$ dBm / 100 MHz" beyaz gürültü olarak modelleniyor, X şirket seçimi. `radar\_takip.py` satır 40'ta X gürültüye ekleniyor. Neden X değişince P_d değişti? Cevabı bu iki yerden yaz.
6. **Varsayım karşılaştırma tablosu:** sütunlar *bizim radar | Blandino IV.A | TR 38.765 Ek A*; satırlar taşıyıcı frekans, SCS, bant, verici gücü, hedef RCS, anten, tespit istatistiği, hedef sayısı. Bizim değerler `radar\_takip.py` PARAM bölümünde (satır 23–32).

Kontrol: bizde ölçülen sonuç (kendi sonucunla karşılaştır)

`tr_metrik_olc.py` çıktısı (Windows, Python 3.11.9; tohum sabit):
P_{f1}: $M = 14$ ($\alpha = \%10$): 2/3000 $\Rightarrow$ %0,07; $M = 70$ ($\alpha = \%50$): 5/3000 $\Rightarrow$ %0,17.
$P_{md} \leq \%5$ için en küçük α: 100 m: %2,9 ($P_d = 1{,}00$); 200 m: %20 (0,99); 300 m: %100 (0,975); 400 m: taranan değerlerin hiçbirinde yok (en yüksek $P_d = 0{,}38$).
300 m, $\alpha = \%20/40/50/100$: $X_{SI} = 40$ dB: 0,18 / 0,56 / 0,73 / 0,96; $X_{SI} = 30$ dB: 1,00 / 1,00 / 1,00 / 1,00. (160 denemede standart hata en çok 0,04.)
Bu betiği Ubuntu'da (senin ortamında) çalıştırmadık; çıktı farklıysa farkı yaz.

Ne teslim edeceksin?

(1) Lin–kod eşleme tablosu; (2) `tr_metrik_olc.py` çıktısı; (3) %90 yüzdelik tablosu ve TR hedefiyle karşılaştırma; (4) X_{SI} açıklaması (3–4 cümle); (5) varsayım karşılaştırma tablosu.

Aziz: ns-3 / 5G-LENA (yük, bant ve Sionna-RT durumu)

Bu görev neden önemli?

ns-3 haberleşme bedelini ölçer. Dickerson modeli kapasitenin $(1 - \alpha)$ ile azalacağını söylüyor; bu kart bunu ns-3'te ölçtürüyor ve sensing'i temsil etme yollarını (bant daraltma, ayrı trafik akışı) ölçümle karşılaştırtıyor. Raporun Yöntem bölümü bu karara bağlı.

Okuma

Kaynak	Yer	Ne arayacaksın	Durum
Ropitault vd. (arXiv 2508.04004), `makaleler/`	III.A, III.B, III.C, IV.A	III.A: 5G-LENA'nın TR 38.901 kanal sınıfları. III.B: `TracesChannelModel` ve `TracesSpectrumPropagationLossModel`, dış ray tracer'ın çok yollu bileşenlerini (gecikme, genlik, faz, AoA/AoD) CSV'den okuyor. III.C: Sionna RT'den CSV üreten boru hattı (`wineslab/sionna-channel-generator`). IV.A: 28 GHz, 100 MHz, tek gNB–UE.	[Okundu]
Dickerson vd. (arXiv 2606.21677)	IV.D	Sabit politikalar: %15 ayırınca throughput sınırı 0,85, %3 ayırınca 0,97. ns-3'te iki politikayı bant daraltmayla taklit edeceksin.	[Okundu]
Capozzi vd., *Downlink Packet Scheduling in LTE* (repo kökünde)	IV.A.2 (Round Robin), IV.B.1 (Maximum Throughput), IV.B.2 (Proportional Fair)	Her algoritmanın metriği; MT'nin zayıf kanallı kullanıcıya ne yaptığı. LTE makalesi; algoritma tanımları için.	[Okundu]
Koutlia vd. (arXiv 2205.03278)	1 (giriş), 5.1	5G-LENA'nın 3GPP referans sonuçlarıyla kalibrasyonu, REM aracı. Rapor için: "simülatör güvenilir mi" cümlesinin kaynağı.	[Okundu]
Gautam vd., *5G Configured Grant Scheduling for ISAC in Vehicular Networks* (2025, CNSM; Consensus)	Özet	ISAC sensing trafiği aperiyodik kabul edilip ns-3 5G-LENA CG modülünde Weibull gelişler ile modellenmiş. "Sensing'i trafik olarak temsil" yolunun literatürde kullanıldığını gösteriyor.	[Yalnızca özet]

Adımlar

1. **Yük taraması** (Ubuntu, repo klasöründe):

```
bash ogretici/betikler/ns3_yuk_sweep.sh      # cikti: /tmp/ns3_yuk.csv
```

`cttc-nr-demo` iki akış kuruyor: akış 1 düşük gecikmeli (`packetSizeUll`, `lambdaUll`), akış 2 en iyi çaba (`lambdaBe`). Betik `--doubleOperationalBand=false` ile ikisini **aynı banda** koyar (varsayılan iki ayrı banttır) ve `lambdaBe`'yi 1000'den 40000'e çıkarır. Tablodan: BE akışının alınan throughput'u hangi gönderilen değerden sonra sabitleniyor?

2. **Bant taraması** (doygun yükte kapasiteyi ölçer):

```
bash ogretici/betikler/ns3_bant_sweep.sh     # cikti: /tmp/ns3_bant.csv
```

$\alpha = 1 - B/50\,\text{MHz}$. Tabloya sütun ekle: $C(\alpha)/C_0$ ve $1 - \alpha$. İki ek ölçüm yap: `--bandwidthBand1=48500000` ($\alpha = \%3$) ve `42500000` ($\alpha = \%15$); Dickerson IV.D'deki iki politikaya karşılık gelir.

3. **İki soru (cevabı kendi ölçümünle ve kodla ver):** (a) 48,5 MHz neden 50 MHz ile aynı throughput'u veriyor? `contrib/nr/examples/cttc-nr-demo.cc` içinde `bandwidthBand1`'in nerede kullanıldığını `grep` ile bul. (b) 10 MHz'de neden hiç paket alınmıyor (`Rx Packets: 0`)? Farklı değerlerle (örn. 15, 20 MHz) sınırı bul.

4. **Sionna-RT örneğinin durumu.** 5G-LENA v5.1'de `cttc-nr-demo-sionna-rt` örneği var (`contrib/nr/RELEASE\_NOTES.md` satır 275–278). `contrib/nr/examples/CMakeLists.txt` içindeki `check_deps(nr_sionna_deps ...)` bloğunu oku: örnek yalnızca `cppyy`, `sionna.rt`, `pybind11` Python paketleri bulunursa ve Windows değilse derleniyor. Bizim WSL'deki `python3` (3.12.3) için şunu çalıştır ve çıktıyı yaz:

```
for m in cppyy sionna.rt pybind11; do python3 -c "import $m" 2>&1 | tail -1; done
cd ~/ehb415/ns-3-dev; ./ns3 run "cttc-nr-demo-sionna-rt --PrintHelp" 2>&1 | tail -3
```

Paket kurmayı **deneme** (sistem Python'una dokunma); yalnızca durumu raporla.

5. **"Hangi yol" notu (1 sayfa).** İki yolu bu kartın ölçümleriyle karşılaştır: (a) bant daraltma (adım 2), (b) sensing'i ayrı trafik akışı olarak ekleme (adım 1, `lambdaBe`). Her yol için: ns-3'te ne ölçüyor, hangi α tanımına

karşılık geliyor, ölçümde ne görüldü. Önerini ve nedenini yaz. Perşembe toplantısında bu notu sen sunacaksın.

Kontrol: bizde ölçülen sonuç (kendi sonucunla karşılaştır)

Adım 1 (`doubleOperationalBand=false`, 2 UE): akış 1 her `lambdaBe` için 10,23–10,24 Mbps ve 0,28–0,32 ms. Akış 2 (BE): gönderilen 10,24 → alınan 10,24; 51,20 → 51,18; 102,40 → 102,35; 204,80 → **164,54** (gecikme 59,35 ms, kayıp %19,66); 409,60 → **164,54** (104,47 ms, kayıp %59,83).
Adım 2 (BE akışı alınan Mbps / gecikme ms): 50 MHz 164,54 / 104,47; 48,5 MHz 164,54 / 104,47; 42,5 MHz 143,77 / 119,85; 40 MHz 133,41 / 130,15; 30 MHz 102,54 / 183,46; 25 MHz 77,06 / 243,99; 10 MHz 0 paket.
Adım 4: `cppyy`, `sionna.rt`, `pybind11`: üçü için de `ModuleNotFoundError`; `./ns3 run cttc-nr-demo-sionna-rt` bizde `Couldn't find the specified program` veriyor.
Varsayılan (`doubleOperationalBand=true`) iki ayrı banttır; bu durumda BE akışı 189,70 Mbps'te doyuyor.

Ne teslim edeceksin?

(1) Yük tablosu; (2) bant tablosu (C/C_0 ve $1-\alpha$ sütunlarıyla); (3) 3. adımdaki iki sorunun cevabı; (4) 4. adımın komut çıktıları; (5) 1 sayfa "hangi yol" notu; (6) 3 cümle: 5G-LENA'nın güvenilirliği (Koutlia).

Fatih: OCUDU (PRS konumu, kod okuma, çok kullanıcı)

Bu görev neden önemli?

OCUDU gerçek gNB kodu. Ölçtüğümüz bir bulgu var: PRS sembol sayısı DL throughput'unu değiştirmiyor. Bu kart nedenini kodda gösteriyor ve yeni bir gözlemi (PRS'in ofsetine göre UL'nin düşmesi) açık soru olarak sana bırakıyor. Cevaplar raporda "OCUDU zamanlayıcısı PRS'e nasıl davranıyor" bölümüne girer.

Okuma

Kaynak	Yer	Ne arayacaksın	Durum
Khosroshahi vd. (arXiv 2408.00667), `makaleler/`	II.A, VI, VII	II.A: PRS comb 2/4/6/12, sembol 1/2/4/6/12; sensing ve iletişim sembolleri aynı RE'yi paylaşamaz. VI: PRS ile PDSCH arasında zaman ve frekans kaynak dağıtımı için optimizasyon problemi (denklem 27a–27e). VII: 28 GHz, 120 kHz için bulunan nokta: PRS'e 6 slot ve 12 PRB, kalan PDSCH'e.	[Okundu]
Lindenschmitt vd. (arXiv 2509.16917)	3.3, 4.3, 4.4, 5	3.3 ve 4.3: radar işlemenin RU, DU ya da DU'ya bağlı RIC'te yapılması; gerçek zamanlılık, fronthaul yükü, maliyet ve güvenlik karşılaştırması. 4.4: dilimleme ile farklı SCS ve TDD düzenlerinin sensing kalitesine etkisi ve zamanlayıcının dengeyi nasıl kuracağı "iyi anlaşılmış değil". 5: ikinci bir "sniffer RU" ile standart değişikliği gerektirmeyen öneri.	[Okundu]
OCUDU kodu (`\textasciitilde/ehb415/ocudu/lib/scheduler/`)	`common\_scheduling/prs\_scheduler.cpp` (satır 26–27, 193)	PRS PDU'sunun sembolleri `[symbol_offset, symbol_offset+nof_symbols)`; satır 193'te bu semboller ve PRB'ler ızgarada `fill` ile dolu işaretleniyor.	[Okundu]
	`ue\_scheduling/intra\_slice\_scheduler.cpp` satır 865–912	`update_used_dl_vrbs`: dolu PRB'ler `common_pdsch_td_resources()[0].symbols` aralığında taranıyor (satır 876–878); kod "Implementation defined" notu taşıyor.	[Okundu]
	`config/time\_domain\_resource\_helper.cpp`	Bu aralığın başlangıcı CORESET'in bittiği sembol, bitişi 14 (`generate_dedicated_pdsch_td_res_list`, `calculate_minimum_pdsch_symbol`).	[Okundu]
	`pdcch\_scheduling/pdcch\_slot\_resource\_allocator.cpp` satır 161–169	PDCCH için ızgarada çakışma kontrolü (`collides`) ve `fill`.	[Okundu; satırlar arandı, tüm fonksiyon okunmadı]

Adımlar

1. **Ofset deneyi** (Ubuntu, repo klasöründe; betik her koşuyu 50 sn ile sınırlıyor). Betiğin yeni 5. argümanı PRS ilk sembol ofsetidir (varsayılan 2):

```
bash ogretici/betikler/ocudu_prs_kos.sh e1 4 4  52 2      # 4 sembol,  52 PRB, ofset 2
bash ogretici/betikler/ocudu_prs_kos.sh e2 4 12 52 2      # 12 sembol, 52 PRB, ofset 2
bash ogretici/betikler/ocudu_prs_kos.sh e3 4 4  52 0      # ofset 0
bash ogretici/betikler/ocudu_prs_kos.sh e4 4 4  52 10     # ofset 10
bash ogretici/betikler/ocudu_prs_kos.sh e5 4 4  52 5      # ofset 5
```

   Her koşuda DL ve UL için `brate` ile `ok` sütunlarını oku (ilk satır ısınmadır; kararlı satırlara bak). e5 hata verirse `/tmp/out_e5.txt`'deki hata satırını yaz.
2. **Soru 1 (kodla cevapla):** DL throughput ofset 2, 0 ve 10 için neden aynı? `prs\_scheduler.cpp` satır 193 ve `intra\_slice\_scheduler.cpp` satır 876–878'i okuyup bir paragraf yaz. İddialarını satır numaralarıyla destekle.
3. **Soru 2 (açık soru):** ofset 0'da DL değişmezken **UL neden düştü** (77,3 → 57,8 Mbps, `ok` 1000 → 750)? Başlangıç noktası: `pdcch\_slot\_resource\_allocator.cpp` satır 161–169. Cevabını **kanıtla**: log (`/tmp/out_e3.txt` ya da `gnb` log düzeyini açarak) ya da kodda gördüğün satır. Kanıt bulamazsan "bulamadım" yaz ve nerelere baktığını sırala.
4. **Yapılandırma reddi:** e5'teki hata mesajını ve `config/periodic\_resource\_sched\_validator.cpp`'yi oku. Hangi ofset değerlerinin kabul edildiğini bulmak için en az iki ofset daha dene (örn. 1, 7) ve tabloya *kabul / ret* yaz. Ret nedenini mesajdan kopyala.

5. **Çok kullanıcılı deney:**

```
bash ogretici/betikler/ocudu_cok_ue.sh u4   4 0 0      # 4 UE, PRS yok
bash ogretici/betikler/ocudu_cok_ue.sh u4p4 4 4 0      # 4 UE, PRS her 4 slotta
```

6. **Mimari notu (yarım sayfa):** Lindenschmitt 3.3/4.3'e göre radar işlemenin RU, DU ya da RIC'te yapılmasının üç artı/eksisini yaz; Khosroshahi VII'deki nokta (PRS'e 6 slot ve 12 PRB) ile bizim ölçtüğümüz PRS kaybını (`ogretici/veri/ocudu_prs_sweep.csv`) yan yana koyan 3 cümle ekle.

Kontrol: bizde ölçülen sonuç (kendi sonucunla karşılaştır)

Adım 1 (DL Mbps / ok; UL Mbps / ok): e1: 51 / 1000; 77,3 / 1000. e2: 51 / 1000; 77,3 / 1000. e3 (ofset 0): 51 / 1000; **57,8 / 750**. e4 (ofset 10): 51 / 1000; 77,3 / 1000. e5 (ofset 5): yapılandırma reddi: `Detected a collision between the NZP-CSI-RS resource (nzp-CSI-RS-ResourceId=4) and the DL-PRS resource (PRS Resource Set Id=0, PRS Resource Id=0) in slot 13 (period 20 slots).` PRS yokken DL 59,0 ve UL 77,3.
Adım 5 (toplam Mbps): 4 UE PRS yok: 60,0 (UE başına 15,0); 4 UE PRS her 4 slotta: 44,0 (UE başına 11,0). 1 UE: 59,0 ve 44,0; 2 UE: 60,0 ve 44,0; 8 UE: 60,8 ve 45,1.

Ne teslim edeceksin?

(1) Ofset tablosu (DL, UL, kabul / ret); (2) Soru 1 paragrafı (satır numaralı); (3) Soru 2 cevabı ya da "bulamadım" ve baktığın yerler; (4) çok kullanıcılı tablo; (5) yarım sayfa mimari notu.